Comparators in Java

A **Comparator** in Java is an interface that is used to order the objects of user-defined classes. A comparator object is capable of comparing two objects of the same class.

Let's take an example of a function that compares obj1 with obj2 :
public int compare(Object obj1, Object obj2):

There are 2 ways of implementing this sort method -

a. We write a custom function on our own, where we define the sorting logic from scratch.
b. Using the Comparator interface. (Better!)

APNA COLLEGE

How to use sort?

Comparator interface is used to order/sort the objects of a user-defined class.

This interface is present in the java.util package and contains 2 methods compare(Object obj1, Object obj2) and equals(Object element).

Using a comparator, we can sort the elements based on data members/properties. For instance, it may be on the basis of name, age, height etc.

Method of Collections class for sorting List elements is used to sort the elements of List by the given comparator.
public void sort(List list, ComparatorClass c)

Internal working of the sort( ) method of Collections class

sort( ) method calls the Compare method of the classes it is sorting.

To compare 2 objects, it asks "Which is greater?"

Compare method returns one of the 3 values : -1, 0, or 1.

nayakchaithra5@gmail.com

-1 -> obj1 is less than obj2

0 -> obj1 is equal to obj2

1 -> obj1 is greater than obj2

It uses this result to then determine if they(obj1 & obj2) should be swapped for their sort.

To define a **Customized Sorting Order**

```
import java.util.ArrayList;
import java.util.Collections;

public class Solution {
   public static void main (String[] args){
       ArrayList<Person> list = new ArrayList<Person>();
       list.add (new Person("Aman", 34));
       list.add (new Person("Akbar", 42));
       list.add (new Person("Anthony", 28));

       Collections.sort (list);
       System.out.println (list);
   }
}

class Person implements Comparable<Person>{
   String name;
   int age;

   Person(String name, int age){
       this.name = name;
       this.age = age;
   }

   /**
    * This function compares 2 objects
    * A person object is passed as an argument
    * and returns negative integer, zero, or a positive integer
    * as this object is less than, equal to, or greater than the specified object.
    */
   @Override
```

nayakchaithra5@gmail.com

```
    public int compareTo(Person person){
        if(this.age == person.age)
            return 0;
        else
            return (this.age < person.age) ? -1 : 1;
    }

    @Override
    public String toString(){
        return this.name + ":" + this.age;
    }
}
```

Lambda Expressions in Java

A lambda expression is a short block of code which takes in parameters and returns a value. Lambda expressions are similar to methods, but they do not need a name and they can be implemented right in the body of a method.

Format:

Comparator<ClassName> comparator = Comparator.comparing(o -> o.property);

Example:

Comparator<Student> comparator = Comparator.comparing(o -> o.age);
Collections.sort(students, comparator);

nayakchaithra5@gmail.com